One Idea In.
13 Merged PRs Out.
Here's the System.

How I use AI for the thinking before the code.
Two layers. Creative exploration, then autonomous execution.

Claude Code | Dev Workflow | Design-First | Solo Dev | Rust

THE PROBLEM

Vague Ideas Don't Ship.

Starting points that aren't actionable yet.

STARTING POINTS

🤔 **"Search recall is 37.5%"**
Exact identifiers and API names get lost in embedding space. But what's the fix?

🤔 **"Knowledge should have a lifecycle"**
Entries shouldn't live forever at the same priority. But how should they decay?

🤔 **"The retriever is a monolith"**
Can't swap components or benchmark alternatives without rewriting everything.

🤔 **"Agents need session history"**
Conversations vanish after each session. But how do you store them without polluting the knowledge base?

These are directions, not tasks. Jumping straight to code from here means rewriting things three times. The gap between "vague idea" and "actionable issue board" is where the real work happens.

LAYER 1: THE CREATIVE PHASE

Explore. Design. Structure.

This is where I learn. Trade-offs, judgment calls, and decisions that Claude can't make alone.

🤔 **Explore the Problem**
Brainstorm approaches with Claude. Pull in latest research. Compare alternatives.

↓

📄 **Write the RFC**
Document the chosen approach. Decisions, trade-offs, rejected alternatives.

↓

🔍 **5-Persona Review on the Design**
SWE, PM, QA + dynamic reviewers validate the spec before any code.

↓

✅ **Generate Structured Issues**
Dependency graphs, phases, acceptance criteria, tracking issues.

The AI landscape moves fast. Claude can surface approaches I hadn't considered and challenge assumptions I'd been carrying. Hard to do manually when the field shifts every few weeks.

LAYER 1 IN ACTION

"Knowledge Should Have a Lifecycle"

From a vague direction to a complete design.

THREE APPROACHES EXPLORED

A: Score-Based Decay

Fixed thresholds. Simple. But no way to promote knowledge that becomes relevant again.

Rejected: too rigid

B: Usage-Driven Tiering

Promotion paths based on access. Flexible. But no decay for unused entries.

Rejected: no forgetting

C: Hybrid + Policies

Composite scoring with configurable forgetting. Both decay and promotion.

Selected: best of both

THE RFC DEFINED

- 5 memory types (Structural, Decisional, Episodic, Analytical, Procedural)
- 4-tier lifecycle (Hot → Warm → Cold → Forgotten)
- Composite retention scoring with hysteresis to prevent oscillation
- Configurable forgetting policies per scope and type

LAYER 1 OUTPUT

1 RFC Became 13 Structured Issues

Not vague TODOs. Each issue has Rust trait definitions, acceptance criteria, and dependency links.

```
Dependency Graph (key dependencies)

#14 Schema Foundation
  └─ #15 Retention Scoring
  └─ #16 Access Tracking → #22 Tier-Aware Retrieval
  └─ #17 ArcSwap HNSW HIGH RISK → #21 Cold Search
  └─ #18 Forgetting Config
  └─ #26 PostgresStore Migration
      └─ #19 GC Worker HIGH RISK
          └─ #20 GC Safeguards
          └─ #23 Observability → #24 #25
```

13	5	2
Issues created	Phases	High-risk flagged

Every issue has a prompt with specific Rust types to implement, files to modify, and test expectations. The tracking issue has the full dependency graph. This came from the design review, not manual planning.

RESEARCH THAT COMPOUNDS

Nothing Gets Lost Between Sessions

Every brainstorm, every decision, every rejected alternative gets persisted. When I revisit a topic, I see what I explored before and what changed since.

❌

Without Persistence

- Re-explore the same topic from scratch
- Forget why an approach was rejected
- Miss when the landscape changed
- Knowledge evaporates between sessions

✅

With Persistent Memory

- See what was explored before
- Understand why decisions were made
- Track what changed since last visit
- Knowledge compounds over time

HOW IT WORKS

I use corvia (the project I'm building) to persist all research. Claude queries the knowledge base at the start of every session. Prior decisions, rejected alternatives, and design rationale are all searchable.

When I revisited the retrieval pipeline three weeks after the initial design, corvia surfaced what I'd already explored and what had changed. The exploration picked up where it left off instead of starting from scratch.

LAYER 2: AUTONOMOUS EXECUTION

Issue In.
Merged PR Out.

I type `/dev-loop 14` and it runs autonomously. The dev-loop is a skill file that Claude Code reads. It orchestrates the full pipeline.

📄 **Fetch Issue + Context**
Pull GitHub issue, query knowledge base for prior decisions

↓

✅ **Validate Design**
Usually confirms "well-defined, proceeding." Acts as a safety gate.

↓

📝 **Plan + Implement**
Break into tasks, implement with subagents, commit after each task

↓

🔍 **5-Persona Review on Code**
Same review system from previous post. Fix loop until clean.

↓

🚀 **E2E Test + PR + Merge**
Integration tests, happy path, edge cases, then open and merge PR

The skill file orchestrates other skills: brainstorming, plan writing, subagent-driven development. When Layer 1 is solid, the brainstorm phase becomes a quick validation, not a creative session.

THE RESULTS

13 Issues.
13 PRs Merged.

The tiered knowledge feature went from RFC to fully shipped through this two-layer system.

1 RFC written	**13** Issues generated
13 PRs merged	**0** Design rewrites

ANOTHER EXAMPLE: BM25 HYBRID SEARCH

"Recall is 37.5%" became 7 issues across 4 phases with measurable ship gates: Source Recall@5 > 50%, MRR > 0.60, p95 latency < 20ms. Parallel phases identified. Implementation not started yet. The board is ready.

Zero rewrites because the design was validated before code existed. The 5-persona review caught flaws in the spec, not in the implementation.

KEY TAKEAWAYS

What Actually Matters

💡 **The thinking is the hard part**

Layer 1 is where I learn. Exploring trade-offs, challenging assumptions, making judgment calls. The implementation is the easy part.

📈 **Design review prevents rewrites**

Reviewing the RFC before coding is why 13 PRs merged with 0 rewrites. Catching a flaw in a spec costs minutes. In code, it costs hours.

🐘 **Research compounds when persisted**

Saving every brainstorm and rejected alternative means revisiting a topic is faster each time. Knowledge grows across sessions, not within them.

⚡ **Well-defined issues execute themselves**

When Layer 1 is thorough, Layer 2 is almost boring. The dev-loop validates, implements, reviews, and merges. That's the point.

Do you have a structured exploration phase, or do you jump straight to code?

Genuinely curious how people structure the thinking before coding.